

MAKALAH
PENGUNAAN STRUKTUR KONTROL PEMILIHAN,
PERULANGAN DAN ARRAY DALAM PEMROGRAMAN
ALGORITMA

Disusun untuk memenuhi tugas mata kuliah Algoritma Dan Struktur Data

Dosen Pengajar

Bu Deasy Permatasari,S.Kom.,M.T



Disusun Oleh:

Annisa Nurul Aulia	F1A250018
Tsania Alifatur Rabbani	F1A250005
Saepul Umam	F1A250009
Aryo Dwi Prabowo	F1A250017

PROGRAM STUDI TEKNIK INFOTMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS AL-GHIFARI

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Allah SWT atas limpahan rahmat dan hidayah-Nya, sehingga makalah berjudul **“Penggunaan Struktur Kontrol Pemilihan, Perulangan dan Array dalam Pemrograman Algoritma”** ini dapat diselesaikan dengan baik. Makalah ini disusun untuk memenuhi tugas mata kuliah Algoritma sebagai bagian dari upaya memahami fondasi pemrograman yang esensial bagi Mahasiswa Informatika.

Melalui makalah ini, kami berharap dapat berkontribusi dalam pengayaan wawasan mengenai pentingnya penguasaan pemilihan kondisional (*if, else*), pengulangan (*for, while, do-while*), dan pengelolaan array sebagai elemen krusial dalam pemecahan masalah komputasi. Konsep-konsep tersebut tidak hanya relevan secara akademis, tetapi juga aplikatif dalam pengembangan software seperti sistem manajemen data mahasiswa, aplikasi simulasi algoritma, hingga otomatisasi tugas berulang dalam lingkungan pemrograman.

Kami menyadari bahwa makalah ini masih jauh dari kata sempurna, mengingat keterbatasan waktu, referensi, dan pengalaman kami sebagai mahasiswa. Untuk itu, segala bentuk kritik, saran konstruktif, maupun koreksi dari pembaca yang berkenan sangat kami harapkan demi perbaikan mutu karya ilmiah berikutnya. Semoga makalah ini bermanfaat bagi pengembangan ilmu pengetahuan, khususnya di bidang informatika dan algoritma pemrograman.

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB 1 PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Rumusan Masalah	4
1.3 Tujuan Penulisan	4
BAB 2 PEMBAHASAN.	5
2.1 Cara mengimplementasikan struktur kontrol pemilihan (<i>if-else</i>) untuk membuat keputusan dalam algoritma	5
2.2 Cara menggunakan perulangan (<i>for, while, do-while</i>) untuk mengulang proses dalam algoritma	8
2.3 Cara menggunakan array untuk menyimpan dan mengolah data dalam algoritma	12
BAB 3 PENUTUP	17
3.1 Kesimpulan	17
3.2 Saran	Error! Bookmark not defined.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Di era transformasi digital saat ini, kemampuan komputer untuk menyelesaikan masalah secara cepat dan akurat bergantung sepenuhnya pada efisiensi algoritma yang disusun oleh manusia. Algoritma bukan sekadar urutan instruksi, melainkan logika dasar yang memungkinkan teknologi menjalankan fungsi kompleks, mulai dari aplikasi sederhana hingga kecerdasan buatan. Dalam mempelajari pemrograman, terdapat tiga pilar fundamental yang harus dikuasai untuk menciptakan program yang dinamis dan efisien yaitu pemilihan (*selection*), perulangan (*looping*), dan larik (*array*).

1. Dinamika Pengambilan Keputusan (*Selection*)

Sebuah program yang kaku tidak akan mampu menangani berbagai kondisi di dunia nyata. Di sinilah struktur pemilihan atau *branching* berperan penting. Dengan menggunakan logika seperti *if-else*, program dapat "berpikir" dan menentukan tindakan yang berbeda berdasarkan input yang diterima. Tanpa struktur pemilihan, perangkat lunak hanya akan berjalan satu arah dan tidak mampu merespons situasi yang variatif.

2. Efisiensi Melalui Otomatisasi (*Looping*)

Salah satu keunggulan utama komputer dibandingkan manusia adalah kemampuannya melakukan tugas repetitif tanpa kelelahan. Struktur perulangan memungkinkan programmer mengeksekusi blok kode berkali-kali hanya dengan beberapa baris instruksi. Hal ini tidak hanya menghemat waktu penulisan kode, tetapi juga mengoptimalkan penggunaan memori dan memastikan konsistensi tugas yang dilakukan secara berulang.

3. Pengorganisasian Data yang Terstruktur (*Array*)

Dalam pengembangan aplikasi skala besar, mengelola banyak data secara individu menggunakan variabel tunggal sangatlah tidak praktis. Array hadir sebagai solusi untuk menyimpan sekumpulan data dengan tipe yang sama dalam satu wadah terorganisir

Penggunaan array mempermudah pengelolaan, pencarian, dan pengolahan data secara masif, yang menjadi fondasi bagi struktur data yang lebih kompleks di tingkat lanjut.

1.2 Rumusan Masalah

1. Bagaimana cara mengimplementasikan struktur kontrol pemilihan (*if-else*) untuk membuat keputusan dalam algoritma?
2. Bagaimana cara menggunakan perulangan (*for*, *while*, *do-while*) untuk mengulang proses dalam algoritma?
3. Bagaimana cara menggunakan array untuk menyimpan dan mengolah data dalam algoritma?

1.3 Tujuan Penulisan

1. Untuk memahami konsep struktur kontrol pemilihan, perulangan, dan array dalam algoritma.
2. Untuk mengimplementasikan struktur kontrol pemilihan, perulangan, dan array dalam algoritma untuk menyelesaikan masalah.
3. Untuk meningkatkan kemampuan dalam merancang dan mengimplementasikan algoritma yang efektif dan efisien

BAB 2

PEMBAHASAN

2.1 Cara mengimplementasikan struktur kontrol pemilihan (*if-else*) untuk membuat keputusan dalam algoritma

Implementasi struktur kontrol pemilihan adalah cara kita memberikan "kemampuan berpikir" pada program untuk menentukan alur kerja berdasarkan kondisi tertentu. Dalam algoritma, pemilihan ini bekerja seperti persimpangan jalan: program akan memeriksa sebuah syarat, lalu memilih jalur mana yang akan dieksekusi.

Perulangan adalah proses mengulang-ulang eksekusi satu statement atau lebih blok statement tanpa henti, selama kondisi yang dijadikan acuan terpenuhi. Biasanya disiapkan variabel untuk iterasi atau variabel penanda kapan perulangan akan diberhentikan.

Struktur pemilihan *if-else* adalah yang paling fleksibel karena dapat menangani berbagai jenis kondisi, mulai dari perbandingan angka hingga logika boolean yang kompleks. Cara kerja dari sistem perulangan *if-else* adalah mengevaluasi ekspresi logika di dalam kurung. Jika hasilnya benar (*true*), blok kode di dalam *if* dijalankan. Jika salah (*false*), program akan melompat ke bagian *else* atau melanjutkan ke kondisi *else if* berikutnya.

Berikut adalah evaluasi mandiri yang disusun secara ringkas dan langsung pada inti pemahaman untuk menguji pemahaman terhadap konsep perulangan

If-else

Pertanyaan 1:

Sebutkan Dan Jelaskan Tiga Jenis Struktur Pemilihan Yang Umum Digunakan?

- a. Pemilihan tunggal (satu kondisi): Hanya ada satu kondisi yang dicek. Jika benar, instruksi di jalankan, jika salah, tidak terjadi apa apa. Contoh *if* (kondisi)
- b. Pemilihan ganda (dua jalur): Memiliki dua jalur alternatif. Jika kondisi benar, jalankan blok A, jika salah, jalankan blok B. Contoh *if-else*.
- c. Pemilihan majemuk (banyak kondisi): Digunakan untuk mengevaluasi banyak kondisi secara berurutan Contoh *if-else*

Pertanyaan 2:

Apa yang di maksud dengan “*Nested Selection*” pemilihan bersarang? berikan satu contoh kasus nyata di mana struktur ini diperlukan!

Nested Selection atau pemilihan bersarang adalah kondisi Dimana terdapat struktur pemilihan di dalam struktur pemilihan lainnya. Jadi, untuk menjalankan suatu instruksi, seseorang harus melewati pemeriksaan kondisi pertama, lalu diperiksa lagi oleh kondisi kedua.

Contoh kasus: Sistem login bank

- a. Kondisi pertama , Memeriksa apakah *username* terdaftar.
- b. Jika terdaftar, masuk ke kondisi kedua, memeriksa apakah *password* sesuai
- c. Jika *password* benar, barulah akses diberikan.

Pertanyaan 3:

Apa perbedaan antara struktur *If then* dengan *If then else*?

"*If-then*" itu seperti perintah satu arah; jika syarat terpenuhi, lakukan sesuatu, tapi jika tidak terpenuhi, program diam saja atau lanjut ke urutan berikutnya. Sedangkan "*If-then-else*" menyediakan pilihan alternatif; jika syarat terpenuhi lakukan aksi A, tapi jika syarat tidak terpenuhi (salah), program sudah punya instruksi cadangan untuk melakukan aksi B.

Pertanyaan 4:

Apakah kita bisa membuat pemilihan yang tidak ada "*Else*"-nya?

Bisa, Itu disebut pemilihan tunggal. Contohnya: "Jika kamu haus, minum air." Kalau kamu tidak haus, kamu tidak perlu melakukan apa-apa dan bisa lanjut melakukan aktivitas lain. Jadi, "*Else*" (pilihan cadangan) itu sifatnya opsional, tidak wajib ada.

Pertanyaan 5:

Buatkan algoritmik dan program Pascal untuk menentukan apakah seseorang diperbolehkan menonton film di bioskop. Syaratnya, jika usia orang tersebut 17 tahun atau lebih, maka tampilkan "Boleh Menonton", jika kurang dari 17 tahun, maka tampilkan "Tidak Boleh Menonton"

- Bahasa Algoritmik:

PROGRAM CekUsiaBioskop

VAR

usia : integer

ALGORITMA:

READ(usia)

IF (usia >= 17) THEN

PRINT("Boleh Menonton")

ELSE

PRINT("Tidak Boleh Menonton")

ENDIF

- Bahasa Program Pascal

program CekUsiaBioskop;

uses crt;

var

usia : integer;

begin

clrscr;

write ('Masukkan usia Anda: ');

readln(usia);

if (usia >= 17) then

begin

writeln('Boleh Menonton');

end

```

else

begin

    writeln('Tidak Boleh Menonton');

end; readln; end.

```

2.2 Cara menggunakan perulangan (*for*, *while*, *do-while*) untuk mengulang proses dalam algoritma

Struktur perulangan atau *looping* adalah mekanisme dalam algoritma yang memungkinkan kita untuk mengeksekusi sekumpulan instruksi secara berulang selama kondisi tertentu terpenuhi. Tanpa perulangan, program akan menjadi sangat panjang dan tidak efisien jika harus menangani ribuan data.

Berikut adalah penjelasan mengenai tiga jenis perulangan utama beserta cara mengimplementasikannya:

1. Perulangan *for* (*Counted Loop*)

Perulangan *for* digunakan ketika kita sudah tahu secara pasti berapa kali proses tersebut harus diulang. Struktur ini sangat efektif untuk menelusuri elemen di dalam Array.

Komponen Utama:

- a. Inisialisasi: Adalah tahap persiapan membuat kondisi awal sebelum melakukan perulangan, misalnya mengisi variabel dengan nilai awal. Tahap ini dilakukan sebelum memasuki bagian perulangan.
- b. Proses: Tahap proses terjadi didalam bagian perulangan Dimana berisi semua proses yang perlu dilakukan secara berulang-ulang.
- c. Iterasi: Merupakan kondisi penjumlahan agar perulangan terus berjalan.
- d. Terminasi/kondisi perulangan: Terminasi Adalah kondisi berhenti dari perulangan, kondisi berhenti sangat penting dalam perulangan agar perulangan dapat berhenti, tidak menjadi perulangan yang tanpa henti.kondisi perulangan adalah kondisi yang dipenuhi oleh kondisi jalannya algoritma untuk masuk kedalam blok perulangan.

2. Perulangan *while* (*Uncounted Loop*)

Perulangan *while* adalah sebuah mekanisme control yang digunakan untuk mengeksekusi sekumpulan perintah berulang kali selama sebuah kondisi tertentu benar. Cirinya adalah sifatnya yang tidak tetap karena jumlah pengulangannya dipengaruhi oleh keadaan yang berubah saat program dijalankan, bukan yang ditentukan sebelumnya. Cara kerja sistem perulangan *for*, *while* dan *do-while* adalah jika kondisi bernilai salah (*false*) sejak awal, maka blok kode di dalamnya tidak akan pernah dijalankan sama sekali.

3. Perulangan *do-while*

Do-while hampir sama dengan *while*, namun perbedaannya terletak pada letak pengecekan kondisi. Pada *do-while*, kondisi dicek di akhir blok perulangan. Perulangan *do while* hampir sama dengan *while* ia akan terus melakukan perulangan selama kondisi masih memenuhi. Instruksi di dalam blok *do* dipastikan akan minimal berjalan satu kali, meskipun kondisinya sudah bernilai salah sejak awal.

Berikut adalah evaluasi mandiri yang disusun secara ringkas dan langsung pada inti pemahaman untuk menguji pemahaman terhadap konsep perulangan *for*, *while* dan *do-while*.

Pertanyaan 1:

Apa perbedaan antara sistem perulangan *for* dan *while* dan kapan harus menggunakannya?

Perulangan *for* digunakan saat jumlah iterasi sudah diketahui secara pasti (*counted loop*), sedangkan *while* digunakan saat jumlah pengulangannya bergantung pada kondisi yang dinamis dan tidak menentu (*uncounted loop*).

Pertanyaan 2:

Apa yang terjadi jika kondisi pada perulangan *while* tidak pernah menjadi *false*? Bagaimana hal ini mempengaruhi program?

Kondisi yang selalu bernilai *true* dalam pernyataan *if* dapat menyebabkan terjadinya perulangan tanpa akhir yang menghentikan alur eksekusi program, mengakibatkan peningkatan penggunaan *CPU* secara drastis, membuat perangkat

menjadi tidak reponsif atau macet dan berpotensi menguras memori hingga akhirnya program bias mengalami kerusakan.

Pertanyaan 3:

Jelaskan bagaimana pengulangan dapat mendukung dalam menjalankan perhitungan matematika yang berulang seperti factorial atau urutan angka!

Pengulangan memungkinkan untuk menjelaskan operasi matematika berulang, seperti mengalihkan angka 1 hingga n untuk mendapatkan factorial, atau menjumlahkan angka dalam urutan, dengan cara yang otomatis dan efisien.

Pertanyaan 4:

Bagaimana perulangan dapat digunakan untuk mencari data tertentu dalam sebuah array atau *list*?

Dengan menggunakan perulangan, kita dapat memeriksa setiap elemen dalam array satu per satu sampai data yang dicari ditemukan atau seluruh array diperiksa.

Pertanyaan 5:

Buatlah program perulangan *for* untuk *for* menurun dari 10 sampai 1!

Bahasa manusia

- a. Langkah satu, liat layar hp baterai menunjukkan angka 10.
- b. Langkah 2: Karena video terus diputar, baterai turun jadi 9.
- c. Langkah 3: Video lanjut lagi, baterai turun jadi 8.
- d. Langkah 4: Terus begitu sampai baterai menyentuh angka 1.
- e. Langkah 5: HP memberikan peringatan "Baterai Lemah!" dan kamu segera mencari charger

- Bahasa Algoritmik

PROGRAM SimulasiBaterai

DEKLARASI

baterai: INTEGER

ALGORITMA

```

FOR baterai <- 10 DOWNT0 1 DO

    CETAK("Status Baterai: ", baterai, "%")

    DELAY(500)

ENDFOR

CETAK("Peringatan: Baterai Lemah!.")

```

- Bahasa Program

```

program FiturBaterai;

uses crt;

var

    baterai: integer;

begin

    writeln;

    { Inisialisasi: 10, Terminasi: 1, Iterasi: otomatis berkurang }

    for baterai := 10 downto 1 do

        begin

            { Proses }

            writeln('Status Baterai: ', baterai, '%');

            delay(500);

        end;

        { Pesan setelah terminasi tercapai }

        writeln;

    writeln('Peringatan: Baterai Lemah!');

    readln;

end.

```

- Output dari program presentasi baterai

Status Baterai: 10%

Status Baterai: 9%

Status Baterai: 8%

Status Baterai: 7%

Status Baterai: 6%

Status Baterai: 5%

Status Baterai: 4%

Status Baterai: 3%

Status Baterai: 2%

Status Baterai: 1%

Peringatan: Baterai Lemah!

2.3 Cara menggunakan array untuk menyimpan dan mengolah data dalam algoritma

Array merupakan struktur data yang sangat krusial dalam algoritma karena memungkinkan kita untuk menyimpan banyak data dengan tipe yang sama di bawah satu nama variabel. Tanpa array, kita harus membuat ribuan variabel untuk ribuan data, yang tentu saja tidak efisien.

Berikut adalah penjelasan mengenai cara menggunakan dan mengolah data menggunakan array.

1. Konsep Dasar Array

Array bekerja dengan sistem **indeks**. Indeks adalah alamat posisi data di dalam array yang selalu dimulai dari **0**. Jika kita memiliki array dengan ukuran 5, maka indeksnya adalah 0 sampai 4.

2. Menyimpan Data ke Dalam Array

Ada dua cara utama untuk memasukkan data ke dalam array:

- a. Inisialisasi Langsung: Memasukkan data saat variabel array pertama kali dibuat.
 - o *Contoh:* `int nilai[3] = {80, 90, 75};`
- b. Melalui Input User (Menggunakan Perulangan): Ini adalah cara yang paling sering digunakan dalam algoritma untuk menangani data yang dinamis.

Pertanyaan 1:

Apa itu array dan bagaimana cara mendeklarasikan array satu dimensi di Pascal?

Array adalah struktur data yang digunakan untuk menyimpan sekumpulan data dengan tipe yang sama dalam memori berurutan. Di Pascal, array satu dimensi dapat dideklarasikan seperti ini:

```
pascal
var
    arr: array[1..10] of Integer;
```

Ini mendeklarasikan sebuah array bernama `arr` yang bisa menyimpan 10 angka bulat dari indeks 1 sampai 10.

Pertanyaan 2:

Bagaimana cara mengisi data ke dalam array dan bagaimana cara mengakses elemen tertentu dalam array tersebut?

Untuk mengisi data ke dalam array, kita dapat menggunakan indeks secara langsung:

```
pascal
arr[1] := 5;
arr[2] := 10;
```

Untuk mengakses elemen tertentu, cukup gunakan indeksinya:

```
pascal
writeln(arr[1]); // menampilkan nilai dari elemen pertama
```

Pertanyaan 3:

Apa perbedaan antara array satu dimensi dan array multidimensi?*

Array satu dimensi memiliki satu indeks dan digunakan untuk menyimpan data dalam satu baris. Sedangkan array multidimensi memiliki lebih dari satu indeks dan digunakan untuk menyimpan data dalam bentuk tabel atau matriks, misalnya array dua dimensi seperti matriks 3x3. Contoh dalam bahasa pascal;

```
var  
  
matrix: array[1..3, 1..3] of Integer;
```

Pertanyaan 4:

Bagaimana cara mencari nilai maksimum dan minimum dari sebuah array di Pascal?

Caranya adalah melakukan iterasi melalui semua elemen array sambil menyimpan nilai maksimum dan minimumnya. Contoh dalam bahasa pascal;

```
var  
  
i, max, min: Integer;  
  
begin  
  
max := arr[1];  
min := arr[1];  
  
for i := 2 to 10 do  
  
begin  
  
if arr[i] > max then  
  
max := arr[i];  
  
if arr[i] < min then  
  
min := arr[i];  
  
end;  
  
writeln('Nilai maksimum: ', max);
```

```
writeln('Nilai minimum: ', min);  
  
end;
```

Pertanyaan 5:

Buatlah program Pascal yang menggunakan array untuk membaca 10 angka dari pengguna, kemudian menampilkan angka terbesar dan terkecil di antara angka-angka tersebut!

Berikut adalah contoh program lengkapnya:

```
pascal  
  
program CariMaxMinArray;  
  
uses crt;  
  
var  
  
    arr: array[1..10] of Integer;  
  
    i, max, min: Integer;  
  
begin  
  
    clrscr;  
  
    writeln('Masukkan 10 angka:');  
  
    for i := 1 to 10 do  
  
        begin  
  
            write('Angka ke-', i, ': ');  
  
            readln(arr[i]);  
  
        end;  
  
        max := arr[1];  
  
        min := arr[1];  
  
        for i := 2 to 10 do  
  
            begin
```

```
if arr[i] > max then  
    max := arr[i];  
if arr[i] < min then  
    min := arr[i];  
end;  
writeln('Angka terbesar adalah: ', max);  
writeln('Angka terkecil adalah: ', min);  
readln;  
end.
```

BAB 3

PENUTUP

3.1 Kesimpulan

Dalam algoritma, hubungan ketiganya dapat dirangkum sebagai berikut:

- a. Array menyediakan efisiensi penyimpanan (mengelompokkan data terkait).
- b. Perulangan menyediakan efisiensi eksekusi (mengotomatisasi tugas repetitif pada data).
- c. Pemilihan menyediakan intelegensi program (memungkinkan pengambilan keputusan berdasarkan data yang ada).

Berdasarkan pembahasan yang telah diuraikan pada bab-bab sebelumnya, dapat disimpulkan bahwa Pemilihan (*Selection*), Perulangan (*Looping*), dan Array (Larik) merupakan pilar utama dalam membangun algoritma yang efektif dan efisien. Ketiganya memiliki peran yang saling melengkapi.

Sinergi antara ketiga elemen ini sangatlah krusial. Perulangan memungkinkan penelusuran elemen di dalam array, sementara pemilihan memberikan logika penyaringan terhadap data yang sedang diproses. Tanpa penguasaan terhadap ketiga konsep ini, seorang pengembang perangkat lunak akan kesulitan dalam menyusun logika pemrograman yang kompleks dan optimal.

3.2 Saran

Dalam penggunaan struktur control pemilihan, perulangan dan array dalam program dan algoritma, masih banyak yang perlu ditingkatkan dan dikembangkan. Oleh karena itu, kami memberikan beberapa saran untuk pengembangan lebih lanjut. Penggunaan struktur control ini dapat lebih efektif dengan menggunakan algoritma yang lebih canggih dan lebih efisien. Dengan demikian, penggunaan struktur kontrol pemilihan, perulangan dan array dapat menjadi lebih efektif dalam pengembangan program dan algoritma.

DAFTAR PUSTAKA

- Adani, M. R. (2020). *Pengenalan Algoritma Pemrograman dan Contoh Penerapannya*. Jakarta: Informatika.
- Kadiv, A. (2017). *Dasar Pemrograman C++ disertai dengan Pengenalan Pemrograman Berorientasi Objek*. Yogyakarta: Andi Offset.
- Munir, R. (2016). *Algoritma dan Pemrograman dalam Bahasa Pascal dan C++*. Bandung: Informatika.
- Supratno. (2008). *Bahasa Program untuk Sekolah Menengah Kejuruan*. Jakarta: Direktorat Pembinaan Sekolah Menengah Kejuruan.

